

# Supplementary II (General Birthday Attack)

In the manuscript, we claim that  $\mathcal{A}_1$  or  $\mathcal{A}_2$  expects to spend less than 36 minutes (derived from  $2^{31}$  hash evaluations at  $T_h = 0.001$  ms) to generate an aggregate signature comprising  $2^{15}$  individual signatures to frame a vehicle in the schemes. This is based on the birthday paradox.

## I. BIRTHDAY ATTACK

In the manuscript, we take the 128-bit security level as an example to clarify the generalized birthday attack.

The 128-bit security level indicates that an adversary needs to perform approximately  $2^{128}$  fundamental operations (e.g., hash evaluations) to break the scheme via brute force. According to the traditional birthday attack,  $\lambda = 256$  is required to achieve 128-bit security. Our experimental results show the computational cost of a hash evaluation  $T_h$  is approximately 0.001 ms. It would take more than  $10^{25}$  years ( $2^{128} \times 0.001$  ms) on our laptop to find values  $x$  and  $x'$ , such that  $\mathcal{H}(x) = \mathcal{H}(x')$ . In contrast, finding  $2^{15}$  inputs  $\{x_i\}_{i=1}^{2^{15}}$ , such that  $\sum_{i=1}^{2^{15}} \mathcal{H}(x_i) = 0$ , requires a runtime of  $0.001 \text{ ms} \times 2^{31} < 36$  minutes. Wagner's attack algorithm is designed to achieve this goal.

## II. WAGNER'S ALGORITHM(THE SUM IS ZERO)

Wagner's algorithm efficiently finds a tuple of  $n = 2^{15}$  random 256-bit hash values whose sum is any given target value by constructing a binary merging tree. We take zero as the target value to explain Wagner's algorithm:

- 1) The algorithm firstly generates  $2^{31}$  independent random hash values. At the leaf level, the algorithm maintains  $2^{15}$  lists, each storing  $2^{16}$  random hash values.
- 2) In the  $j$ -th level of the tree, two child lists are combined by computing the pairwise sum of their elements, and only those sums whose lower  $16j$  bits are zero are retained to form the parent list.
- 3) This level-by-level merging and filtering procedure is repeated for 15 rounds, gradually fixing lower bits of the accumulated sum to zero.
- 4) After 15 levels, the root list consists of values whose lower 240 bits are all zero and only the upper 16 bits remain random.
- 5) Finally, the algorithm selects a value  $h_1$  such that  $h_1 = 0$  from the  $2^{16}$  hash values.

## III. ANALYSIS

In Step 1), the algorithm computes  $2^{31}$  hash values. According to our experimental results, the execution time of a general hash operation (SHA-1) is almost 0.001 ms. The execution environment is described in the manuscript.  $0.001 \text{ ms} \times 2^{31} < 36$  minutes. The execution time of additive and searching operations is small enough to be negligible.

In Step 2), each sum has its lowest 16 bits equal to zero with probability  $\frac{1}{2^{16}}$ . As each list stores  $2^{16}$  random hash values, the total number of pairs from two lists is  $(2^{16})^2 = 2^{32}$ . Therefore, the expected number of valid sums with the lowest 16 bits equal to zero is exactly  $\frac{2^{32}}{2^{16}} = 2^{16}$ .

After 15 repetitions of Step 2), we obtain a list with  $2^{16}$  hash values in Step 4).

Step 5) finds a 16-bit hash value  $h_1=0$  from  $2^{16}$  random integers. The expected number of  $h_1$  is  $\frac{2^{16}}{2^{16}} = 1$ .

## IV. EXPERIMENT

We run the implementation of the Wagner algorithm on an Ubuntu 24 system equipped with a single-core 1.80-GHz processor and 2-GB of random access memory. In the manuscript, we claimed "Using 256-bit hash values for the attack, it is over  $10^{25}$  years using our laptop to expect to find values  $x$  and  $x'$ , where  $\mathcal{H}(x) = \mathcal{H}(x')$ . In contrast, finding  $2^{15}$  inputs  $\{x_i\}_{i=1}^{2^{15}}$ , where  $\sum_{i=1}^{2^{15}} \mathcal{H}(x_i) = 0$ , requires less than 36 minutes runtime."

Using 256-bit hash values for the attack is excessively resource-consuming, which is computationally infeasible for our experimental setup. Therefore, the source code (Test0.c) uses 64-bit hash values as an example. A 64-bit hash value typically provides 32-bit security, as indicated by the calculation:  $0.0001 \text{ ms} \times 2^{32} > 4000$  seconds. According to Wagner's birthday attack (a variant of the birthday attack), finding 128 hash values  $h_3^{i*}$  that satisfy  $\sum_{i=1}^{128} h_3^{i*} = 0$  requires 128 lists, each containing 256 hash values. Consequently, this setup only provides  $7 + 8 = 15$ -bit security, with a total execution time of  $0.0001 \text{ ms} \times 2^{15} < 33$  ms. However, according to our experimental results, the execution time of each Wagner's birthday attack is 1-4 seconds. We successfully found 10 valid groups. The number of reruns required for each group is:

1, 13, 2, 6, 3, 18, 7, 5, 17, 3

According to the simulations, the average number of reruns for each valid group is 7.5. It is non-negligible.

Please check Test0.c for the source code, and Supplementary III(Wagner 128-Sum 64-bit Hash Summation Algorithm) for the manual of the source codes.

## V. WAGNER'S ALGORITHM(THE SUM IS A HASH VALUE)

In the analysis of Xu's scheme (Subsection 3.2 and 3.3), we claimed that  $\sum_{i=1}^n h_3^{i*} = h_4^*$ . Wagner's algorithm can solve it, too.

- 1) The algorithm firstly generates  $2^{31}$  independent random hash values. At the leaf level, the algorithm maintains  $2^{15}$  lists, each storing  $2^{16}$  random hash values.
- 2) Generate a hash value  $h_4^*$ .
- 3) In the  $j$ -th level of the tree, two child lists are combined by computing the pairwise sum of their elements. The

sums of the first list whose lower  $16j$  bits are identical to the lower  $16j$  bits of  $h_4^*$ , and the sums of other lists whose lower  $16j$  bits are zero are retained to form the parent list.

- 4) This level-by-level merging and filtering procedure is repeated for 15 rounds.
- 5) After 15 levels, the root list consists of values whose lower 240 bits are the lower 240 bits of  $h_4^*$ , and only the upper 16 bits remain random.
- 6) Finally, the algorithm selects a value  $h_3^*$  such that  $h_3^* = h_4^*$  from the  $2^{16}$  hash values.  $h_3^*$  is the sum of 128 random hash values.

The theoretical computational cost is identical to that in Section III.

The source code is provided in `TestH.c`. We successfully found 10 valid groups. The number of reruns required for each group is:

5, 3, 2, 1, 18, 5, 3, 11, 2, 18

According to the simulations, the average number of reruns for each valid group is 6.8. It is non-negligible.

Please check `TestH.c` for the source code, and Supplementary III(Wagner 128-Sum 64-bit Hash Summation Algorithm) for the manual of the source codes.

Please refer to [Wagner2002] for more details on the generalized birthday attack.

[Wagner2002] D. Wagner, "A Generalized Birthday Problem," in *Advances in Cryptology — CRYPTO 2002*. *CRYPTO 2002*, Lecture Notes in Computer Science, vol. 2442, pp 288–304, 2002.